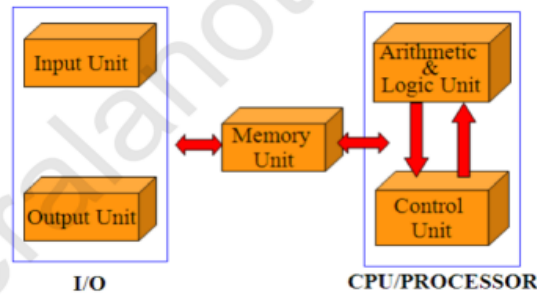


MODULE 1

1. Explain the functional units of computer



- **Input Unit:** Data/ instructions are fed to a computer through input unit
Examples: keyboard, mouse. Scanner, joystick
- **Arithmetic & Logic Unit:** ALU consist of necessary logic circuits like adder, comparator etc., to perform arithmetic and logic operations such as addition, multiplication, comparison of two numbers etc.
- **Control Unit:** Control unit co-ordinates activities of all units by issuing control signals. Control signals issued by control unit govern the data transfers and then appropriate operations take place. Control unit interprets or decides the operation/action to be performed.
- **ALU & Control Unit together called Central Processing Unit (CPU).** CPU is the brain of a computer system.
- **Memory Unit:** Memory unit stores the program instructions (Code), data and results of computations etc
Examples: ROM, RAM
- **Output Unit:** Computer after computation returns the computed results, error messages, etc. via output unit.
Examples: Monitor, printer, speaker

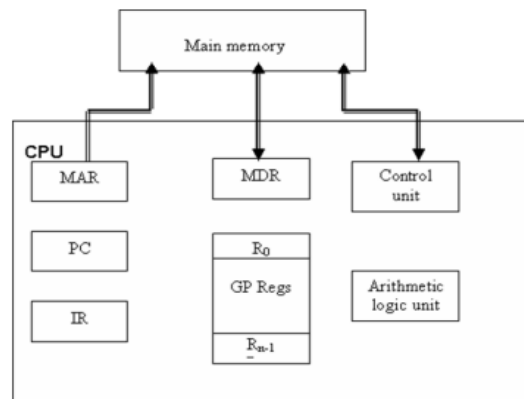
2. Describe the operational concepts of a computer.

- A set of instructions called a program reside in the main memory of computer. Data is also stored in the memory
- The CPU fetches those instructions sequentially one-by-one from the main memory, decodes them and performs the specified operation on associated data operands in ALU.
- Processed data and results will be stored in the memory and displayed on an output unit.
- All activities pertaining to processing and data movement inside the computer machine are governed by control unit.

3. What is a register? Describe the functions of various registers in CPU with a neat diagram./ Explain the internal architecture of a processor.

Registers are high-speed storage area within the CPU. All data must be stored in a register before it can be processed.

In addition to ALU CPU also contains CPU also contains Instruction Register (IR), the Program Counter(PC), the general-purpose registers, the Memory Address Register(MAR) and Memory Data Register(MDR).



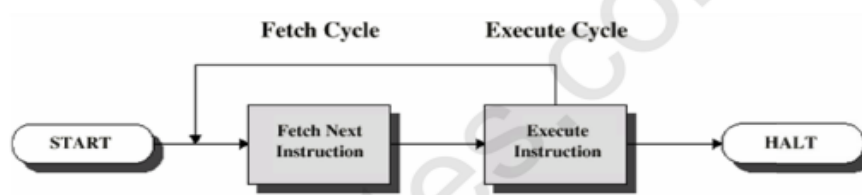
Functions

- **Instruction Register(IR):** contains the instruction that is being executed. Its output is available to the control circuits, that generate the timing signals for control of the actual processing circuits needed to execute the instruction.
- **Program Counter(PC):** It contains the memory address of the next instruction to be fetched and executed
- **Memory Address Register (MAR):** holds the address of the memory location to or from which data is to be transferred.
- **Memory Data Register(MDR):** contains the data to be written into or read-out of the addressed memory location.
- **General- purpose Registers:** are used for holding data, intermediate results of operations. They are also known as scratch-pad registers.

4. Illustrate the basic operational concepts in transferring data between main memory and processor.

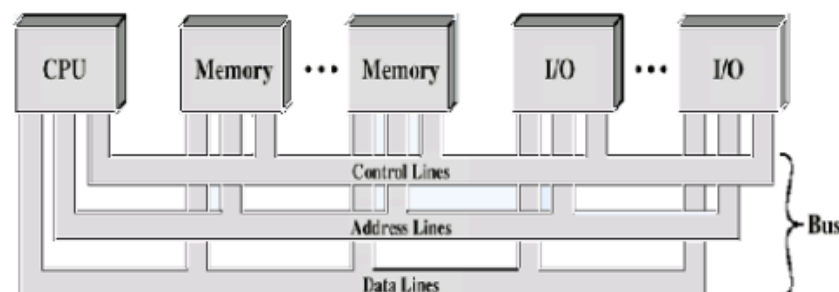
- Programs reside in the memory & usually get these through the input unit.
- Execution of the program starts when the PC is set to point at the first instruction of the program.
- Contents of PC are transferred to MAR and a Read Control Signal is sent to the memory.
- The addressed word is read out of the memory and loaded into the MDR.
- Now contents of MDR are transferred to the IR & now the instruction is ready to be decoded and executed.

- If the instruction involves an operation by the ALU, it is necessary to obtain the required operands.
- An operand in the memory is fetched by sending its address to MAR & Initiating a read cycle.
- When the operand has been read from the memory to the MDR, it is transferred from MDR to the ALU.
- After one or two such repeated cycles, the ALU can perform the desired operation.
- If the result of this operation is to be stored in the memory, the result is sent to MDR.
- Address of location where the result is stored is sent to MAR & a write cycle is initiated.
- The contents of PC are incremented so that PC points to the next instruction that is to be executed.



5. List different types of buses used to interconnect various functional units in a computer.

- **Data Bus:** It is used for transmission of data. The number of data lines corresponds to the number of bits in a word.
- **Address Bus:** It carries the address of the main memory location from where the data can be accessed.
- **Control Bus:** It is used to indicate the direction of data transfer and to coordinate the timing of events during the transfer.



6. Explain zero, one, two and three address instruction with an example for each.

Three -address instruction format

This instruction format consists of three addresses along with an operation field. The three addresses include the address of the first operand, address of the second operand, address to store the result.

Format: Operation code Source1,source2, destination

Example: Add A,B,C

$$[C] \leftarrow [A] + [B]$$

Two -address instruction format

This instruction format consists of two addresses along with an operation field. The two addresses include the address of the first operand, address of the second operand; the result is stored in one of the operand address.

Example: Add A, B

$$B \leftarrow [A] + [B]$$

One -address instruction format

This instruction format consists of one address along with an operation field. The address is that of the first operand. The second operand and the result are stored in a CPU register called **accumulator**. A machine has only one accumulator; it need not be explicitly mentioned in the instruction.

Example: Add A

Zero -address instruction format

A **stack** is included in the CPU for performing arithmetic and logic instructions with no addresses. The operands are pushed onto the stack from memory and ALU operations are implicitly performed on the top elements of the stack.

Example: Add

Top of stack = top of stack + second top of stack

7. What is an addressing mode? List different addressing modes used in an instruction execution.

The address generated by the CPU in-order to access the operand in the memory is termed as an effective address. The methods used to provide an access path to operands in memory and CPU registers is addressing mode.

Various addressing modes are

- Immediate
- Register
- Direct(Absolute)
- Indirect
- Index (Displacement)
- Base with index
- Base with index and offset
- Relative
- Auto increment
- Auto decrement

8. Describe auto increment addressing mode with the help of an example.

After accessing the operand the content of this register is automatically incremented to point the next item in the list.

Increment R_i

Add $(R1)+, R2$

9. Identify the following addressing modes.

Move #500,R0-Immediate Addressing mode

Add 4(R0),R1-Index addressing mode

Add #16,R0-Immediate addressing mode

Add 12(R0),R3-Index addressing mode

Move R1,Sum1-Combination of register and direct

Add [R2],[R0]- Indirect addressing mode

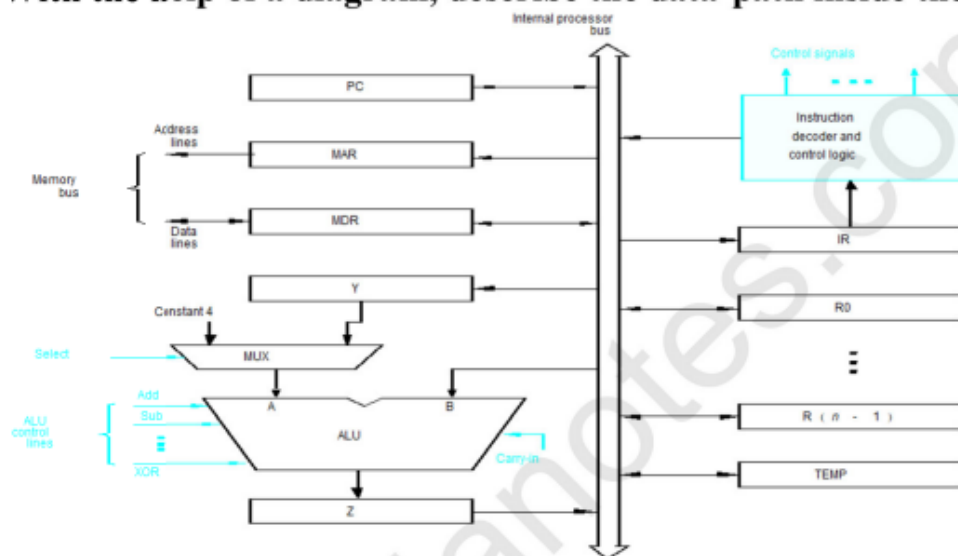
Add (R2)+,R0-Autoincrement mode

Add Loc,R2- Direct addressing mode

Add (R3),R2-Indirect addressing mode

Add (R5),R1-Indirect addressing mode
 Move R5, R4-Register Addressing mode
 Move Loc1,Loc2-Direct addressing mode
 Load 20(R1),R5-Index addressing mode
 Store R5,30(R1,R2)-Base with index and offset
 Sub (R1)+,R5-Autoincrement mode
 Add (-R1),R5-Autodecrement mode
 Jump 10(PC)- Relative addressing mode

10. With the help of a diagram, describe the data-path inside the processor.



- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (fetch phase).

$$IR \leftarrow [[PC]]$$

- Assuming that the memory is byte addressable(4 byte long), increment the contents of the PC by 4 (fetch phase).

$$PC \leftarrow [PC] + 4$$

- PC Keeps track of execution of a program
- Contains the memory address of the next instruction to be fetched and executed.

- Constant 4 is used by the processor to increment the contents of PC.
- Carry out the actions specified by the instruction in the IR (execution phase).
- The data and address lines of external memory connected to the processor bus via MDR and MAR
- Register MDR has 2 inputs and 2 outputs.:
 - Contains data to be written into or read out of the addressed location.
 - Data can be loaded into MDR either from memory bus or from internal processor bus.
- MAR Holds the address of the location to be accessed.
 - I/P of MAR is connected to Internal bus and an O/p to external bus.
- The instruction decoder and control logic circuit is responsible for implementing the actions specified by the instruction loaded in IR
 - The decoder generate the control signals needed to select the registers
 - The registers, ALU and interconnecting bus are collectively referred to as the data path.
- MUX
 - Select either the output of the register Y or a constant value 4 to be provided as input A of the ALU.

11. Give the control sequence for execution of instruction Add[R3],R1 using a single bus organization

Step	Action
1	PC _{out} , MAR _{in} , Read, Select4,Add, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMF C
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMF C
6	MDR _{out} , SelectY,Add, Z _{in}
7	Z _{out} , R1 _{in} , End

12. Write down the sequence of actions needed to fetch and execute the instruction

LOAD 10(R2),R1.

1. Pcout, MARin, Read, Select4, Add, Zin
2. Zout, Pcin, Yin, WMFC
3. MDRout, IRin
4. R2out, MARin, Read
5. WMFC, MDRout
6. IR10out, Yin
7. Select Y, Add, Zin
8. Zout, R1 in, End

13. Write down the sequence of actions needed to fetch and execute the instruction

STORE R1, 10(R2)

1. Pcout, MARin, Read, Select4, Add, Zin
2. Zout, Pcin, Yin, WMFC
3. MDRout, IRin
4. R2out, MARin, Read
5. WMFC, MDRout
6. IR10out, Yin
7. Select Y, Add, Zin
8. Zout, MARin
9. R1 out, MDRin, Write
10. WMFC, End

14. Give the sequence of control steps required to perform the operation ADD NUM, R1

1. Pcout, MARin, Read, Select4, Add, Zin
2. Zout, Pcin, Yin, WMFC
3. MDRout, IRin
4. IRNUMout, Yin,
5. SelectY, R1out, Add, Zin
6. Zout, R1in, End

15. Give the sequence of control steps to perform the operation ADD (NUM),R1

1. Pcout, MARin, Read,Select4,Add, Zin
- 2.Zout, Pcin, Yin,WMFC
- 3.MDRout, IRin
4. IRNUMout,MARin, Read
5. WMFC, MDRout,MARin,Read
6. WMFC, MDRout,Yin
7. Select Y, R1out, Add,Zin
8. Zout,R1in,END

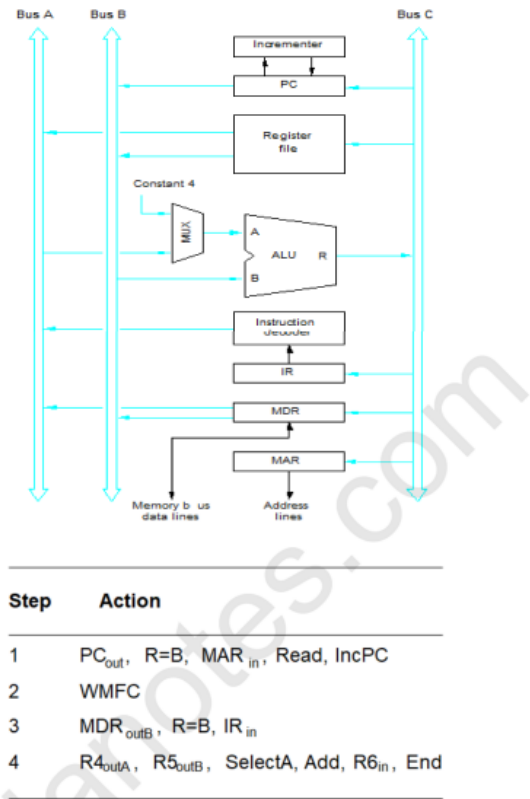
16. Give the sequence of control steps required to perform the operation ADD R1, NUM

1. PCout, MARin, Read,Select4,Add, Zin
- 2.Zout, Pcin, Yin,WMFC
- 3.MDRout, IRin
4. IRNUMout, MARin, Read
5. WMFC, MDRout, Yin,
6. Select Y, R1out,Add, Zin
7. Zout, MDRin, Write
8. WMFC, END

17. Illustrate the advantages of using multiple bus organization over single bus organization.

- Multiple Bus Organization Improves Efficiency.
- Additional Buses Allow Expansion.
- More Buses Means More Compatibility
- Processor speed is high in multiple bus organization

18. Draw the diagram of a multi-bus organization with 3 buses. Write the control sequence for the instruction Add R4, R5, R6 for the above mentioned multi-bus organization.



19. How is a branching instruction executed?

A branch instruction replaces the contents of PC with the branch target address, which is usually obtained by adding an offset X given in the branch instruction. The offset X is usually the difference between the branch target address and the address immediately following the branch instruction

Control Sequence of a branch instruction

1. PC out, MKAR in, Read, Clear Y, Set carry in to ALU,ADD,Zin
2. Z out, PC in, WMFC
3. MDR out, IR in
4. PC out, Y in
5. Offset field of IR out, ADD, Zin
6. Zout, PC in ,End

20. Compare auto increment and auto decrement addressing modes with examples

Ans:

Auto increment Mode = after operand addressing , the contents of the register is incremented.eg: (R4)+

Decrement Mode = before operand addressing, the contents of the register is decrement.eg: -(R4)

21. Outline the steps involved in the execution of an instruction.

Ans: Module 1 – Page 20

22. Name the registers which are connected to both external and internal bus? What are the signals associated with these registers?

Ans:

MAR, MDR - (Write full form)

1) MAR_{in} ,

2) MDR_{in} , MDR_{out} , MDR_{inE} , MDR_{outE} ,

23.

Identify the addressing modes that can be used for representing the following higher level language constructs in machine level. Illustrate each addressing mode using an example.

1) Arrays

2) Pointers

3) Constants

4) Variables

Ans :

1) Arrays - indexed addressing mode

2) Pointers - indirect addressing mode

3) Constants - Immediate addressing mode

4) Variables - Register Mode, Absolute mode

(Write correct answer with example for each)

24.

Illustrate the single bus organization of processor unit with the help of suitable diagrams. Explain, listing the control signals, how the following operations are handled in this organization.

- i) Transfer contents of register R5 to R1
- ii) Move (R6), R2 (*Fetch a word from memory and move it to register R2, when the memory address is stored in register R6*)

Ans:

Illustrate the single bus organization of processor unit with the help of suitable diagrams.

Explanation with diagram - 4 marks

Explain , listing the control signals, how the following operations are handled in this organization.

- i) Transfer contents of register R5 to R1*

Set output enable signal of R5 to high. This places contents of R5 on the bus. Next set input enable signal of R1 to high. This loads data from processor bus to register R1.

- ii) Move (R6), R2 (Fetch a word from memory and move it to register R2, when the memory address is stored in register R6)*

R6_{out}, MAR_{in}, Read

MDR_{inE}, WMFC

MDR_{out}, R2_{in}

25. Compare and contrast single bus and multi-bus organization of CPU?

Single bus organization:

Allows only one transfer at a time.

It costs very low

It is flexible for attaching peripheral devices.

Its performance is low

Multiple bus organization:

Allows two or more transfer at a time.

It costs high

It provides concurrency in operation.

Its performance is high

26.

Write the three-address, two-address and one-address representations of the operation below with relevant assumptions, evaluate following:

i, $(A+B) * (C+D)$

ii, $C \leftarrow [A] + [B]$

Ans:

$(A+B) * (C+D)$

● Three-Address

1. ADD A, B, R1 ; $R1 \leftarrow M[A] + M[B]$
2. ADD C, D, R2 ; $R2 \leftarrow M[C] + M[D]$
3. MUL R1, R2, X ; $M[X] \leftarrow R1 * R2$

● Two-Address

1. MOV A, R1 ; $R1 \leftarrow M[A]$
2. ADD B, R1 ; $R1 \leftarrow R1 + M[B]$
3. MOV C, R2 ; $R2 \leftarrow M[C]$
4. ADD D, R2 ; $R2 \leftarrow R2 + M[D]$

5. MUL R2, R1 ; $R1 \leftarrow R1 * R2$

6. MOV R1, X ; $M[X] \leftarrow R1$

● One-Address [AC- ACCUMULATOR]

1. LOAD A ; $AC \leftarrow M[A]$
2. ADD B ; $AC \leftarrow AC + M[B]$
3. STORE T ; $M[T] \leftarrow AC$
4. LOAD C ; $AC \leftarrow M[C]$
5. ADD D ; $AC \leftarrow AC + M[D]$
6. MUL T ; $AC \leftarrow AC * M[T]$
7. STORE X ; $M[X] \leftarrow AC$

$C \leftarrow [A] + [B]$

- Three-Address

1. ADD A, B, C ; $C \leftarrow M[A] + M[B]$

- Two-Address

1. MOV A, R1 ; $R1 \leftarrow M[A]$

2. ADD B, R1 ; $R1 \leftarrow R1 + M[B]$

3. MOV R1, C ; $M[C] \leftarrow R1$

- One-Address [AC- ACCUMULATOR]

1. LOAD A ; $AC \leftarrow M[A]$

2. ADD B ; $AC \leftarrow AC + M[B]$

3. STORE C ; $M[C] \leftarrow AC$

27. Outline the differences in instruction execution during straight line sequencing and branching using suitable examples.

Ans: Module 1 – Page 9 , 10 , 11

28.

- With the help of a neat figure, describe the data path inside the processor?
- Draw the diagram of single bus organization, write the control sequence for the instruction ADD [R2],R3 for the above mentioned single bus organization.

Ans:

a.

The figure 2 shows how the memory and the processor can be connected. In addition to the ALU and the control circuitry, the processor contains a number of registers used for several different purposes.

The *instruction register (IR)* holds the instruction that is currently being executed. The *program counter (PC)* contains the memory address of the next instruction to be fetched and executed. In addition to the IR and PC, general-purpose registers R_0 through R_{n-1} , often called *processor registers*. They serve a variety of functions, including holding operands that have been loaded from the memory for processing.

Operating Steps

- Programs reside in the memory through input devices.
- PC is set to point to the first instruction.
- The contents of PC are transferred to MAR. A read signal is sent to the memory.
- The first instruction is read out and loaded into MDR.
- The contents of MDR are transferred to IR.
- Decode and execute the instruction. Get operands for ALU (Address to MAR – Read – MDR to ALU).
- Perform operation in ALU and Store the result back to general-purpose register.
- Transfer the result to memory (address to MAR, result to MDR – Write).
- During the execution, PC is incremented to the next instruction.

In addition to transferring data between the memory and the processor, the computer accepts data from input devices and sends data to output devices.

b.

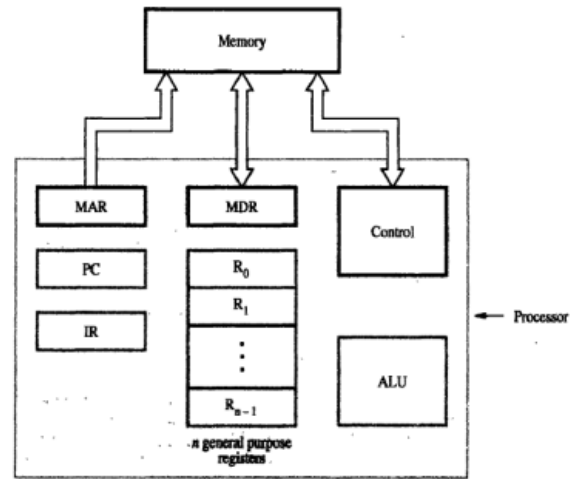


Figure 1.2 Connections between the processor and the memory.

The ALU is a combinational circuit that has no internal storage. It performs arithmetic and logic operations on the two operands applied to its A and B inputs, one of the operands is the output of the multiplexer MUX and the other operand is obtained directly from the bus. The result produced by the ALU is stored temporarily in register Z.

Therefore, a sequence of operations to add the contents of register R1 to those of register R2 and store the result in register R3 is

1. $R1_{out}, Y_{in}$
2. $R2_{out}, \text{Select } Y, \text{Add}, Z_{in}$
3. $Z_{out} R3_{in}$

Step 1: The output of register R1 and the input of register Y are enabled, causing the contents of R1 to be transferred over the bus to Y.

Step 2: The multiplexer's Select signal is set to SelectY, causing the multiplexer to gate the contents of register Y to input A of the ALU. At the same time, the contents of register R2 are gated onto the bus and, hence, to input B. The function performed by the ALU depends on the signals applied to its control lines. In this case, the Add line is set to 1, causing the output of the ALU to be the sum of the two numbers at inputs A and B. This sum is loaded into register Z because its input control signal is activated.

Step 3: The contents of register Z are transferred to the destination register, R3. This last transfer cannot be carried out during step 2, because only one register output can be connected to the bus during any clock cycle.

Fetching a Word from Memory

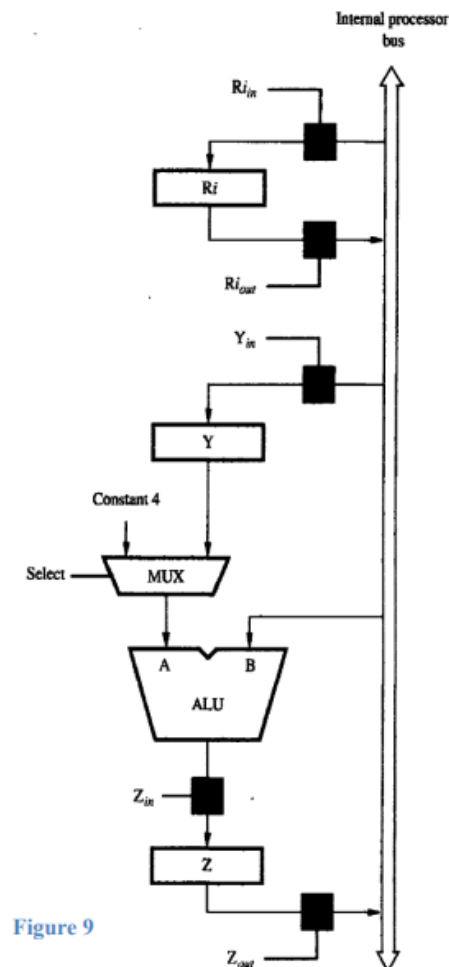


Figure 9

29.

Differentiate between big endian and little endian byte ordering.

Consider a computer that has a byte addressable memory organized as 32 bit words. A program reads ASCII characters entered at a key board and stores them in successive byte locations, starting at location 1000. Show the contents of the two memory words at locations 1000 and 1004 after the name *Johnson* has been entered in case:

- i) Big Endian Byte ordering is used
- ii) Little Endian Byte ordering is used

(ASCII equivalent of characters in *Johnson* in hex will be 4A, 6F, 68, 6E, 73, 6F, 6E. You can indicate unused byte locations using XX).

Ans:

Solution:

Let's examine the contents of memory locations 1000 and 1004 after the name "Johnson" has been entered.

i) Big Endian: memory contents will be as follows:

Location 1000: 4A (J) Location 1001: 6F (o) Location 1002: 68 (h) Location 1003: 6E (n) Location 1004: 73 (s) Location 1005: 6F (o) Location 1006: 6E (n) Location 1007: XX (unused)

The memory word at location 1000 has 4A 6F 68 6E, and in 1004 has 73 6F 6E XX.

ii) Little Endian: memory contents will be as follows:

Location 1000: 6E (n) Location 1001: 68 (h) Location 1002: 6F (o) Location 1003: 4A (J) Location 1004: XX (unused) Location 1005: 6E (n) Location 1006: 6F (o) Location 1007: 73 (s) The memory word at location 1000 has 6E 68 6F 4A, and in 1004 has XX 6E 6F 73.

Please note that the XX represents unused byte locations, as the memory is organized as 32-bit words.

30. Explain how addressing modes contribute to the efficiency and flexibility of computer architecture.

Ans:

The addressing modes help us specify the way in which an operand's effective address is represented in any given instruction.

These can also be said as the advantages of using addressing mode: To give programming versatility to the user by providing such facilities as pointers to memory, counter for loop control, indexing of data, and program relocation. To reduce the number of bits in the addressing field of the instruction.

31.

Draw the block diagram of a single-bus organization and outline the control sequence for the instruction SUB (R2), R3 in this organizational structure. Provide an explanation of each step in the control sequence.

32.

In the context of addressing modes, elaborate on the concept of indirect addressing. Provide examples and explain how it contributes to the flexibility and versatility of instruction sequences in computer architecture.

33. Define three, two and one-address instructions with one example for each.

34. Why is the Wait-for-Memory-Function-Completed step required when performing memory transfer operations?

Ans: . Wait for the MFC(Memory Function Completed) response from the memory

35. With a neat diagram, explain the basic operational concepts in transferring data between the processor and the main memory.

36. What are addressing modes? Explain the different types of addressing modes with examples

37.

Draw the diagram of a single-bus organization of the datapath inside a processor. Give the control sequence for executing the instruction $\text{Add } [R1], R2$ ie, $[R1] \leftarrow [R1] + R2$ in a single-bus organization.

38. Give the control sequence for implementing the conditional branch instruction Branch-on-Negative in a single bus processor organization.

39. With a neat diagram explain the internal architecture of CPU?

40. Enumerate the sequence of actions (control signals) involved in executing an unconditional branch instruction?

Step	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMFC
3	MDR_{out} , IR_{in}
4	Offset-field-of- IR_{out} , Add, Z_{in}
5	Z_{out} , PC_{in} , End

41. What do you mean by addressing modes, with proper examples explain in detail about various address modes?

42. Write down the sequence of actions needed to fetch and execute instruction: STORE $R1, [R2]$

43. Draw the diagram of a multi-bus organization with 3 buses, write the control sequence for the instruction

ADD $[R1], R2, R3$

for the above mentioned multi-bus organization.

44. What are condition codes, list the different condition codes?

Ans:

Condition Codes

The processor keeps track of instruction about the results of various operations for use by subsequent conditional branch instructions. This is accomplished by recording the required information in individual bits often called as **conditional code flags**. These flags are usually grouped together in a special processor register called the **condition code register** or **status register**. Individual condition codes are set to 1 or cleared to 0, depending upon the outcome of the operation performed.

Four commonly used flags are

- N (negative)** Set to 1 if the result is negative; otherwise, cleared to 0
- Z (zero)** Set to 1 if the result is 0; otherwise, cleared to 0
- V (overflow)** Set to 1 if arithmetic overflow occurs; otherwise, cleared to 0
- C (carry)** Set to 1 if a carry-out results from the operation; otherwise, cleared to 0

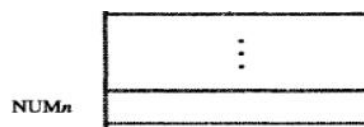


Figure 2.10 Using a loop to add n numbers.

45. Differentiate between Big Endian and Little Endian.

Big-endian and little-endian are terms that describe the order in which a sequence of bytes are stored in computer memory. Big-endian is an order in which the "big end" (most significant value in the sequence) is stored first (at the lowest storage address). Little-endian is an order in which the "little end" (least significant value in the sequence) is stored first. For example, in a big-endian computer, the two bytes required for the hexadecimal number 4F52 would be stored as 4F52 in storage (if 4F is stored at storage address 1000, for example, 52 will be at address 1001). In a little-endian system, it would be stored as 524F (52 at address 1000, 4F at 1001).

In big endian, you store the most significant byte in the smallest address. Here's how it would look:

Address	Value
1000	90
1001	AB
1002	12
1003	CD

In little endian, you store the *least* significant byte in the smallest address. Here's how it would look:

Address	Value
1000	CD
1001	12
1002	AB
1003	90

46.

Give the control sequence for execution of instruction *Add[R3],R1*

Step	Action
1	PC _{out} , MAR _{in} , Read, Select4, Add, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMF C
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMF C
6	MDR _{out} , SelectY, Add, Z _{in}
7	Z _{out} , R1 _{in} , End

47.

1. Write the Sequence of control signal to add the content of register R1 to those of register R2 and store the result in R3 at the execution phase.

Ans :

1. R_{1out}, Y_{in}
2. $R_{2out}, Select Y, Add, Z_{in}$
3. Z_{out}, R_{3in}

48.

2. Write a short note on a. IR b. PC c. MAR d. MDR

Ans :

Instruction Register (IR):

- It holds the instruction that is currently being executed.
- It generates the timing signals.

Program Counter (PC):

- It contains the memory address of the next instruction to be fetched for execution.

Memory Address Register (MAR):

- It holds the address of the location to be accessed.

Memory Data Register (MDR):

- It contains the data to written into or read out of the address location.
- MAR and MDR facilitates the communication with memory.

49.

Explain single bus organization with help of figure

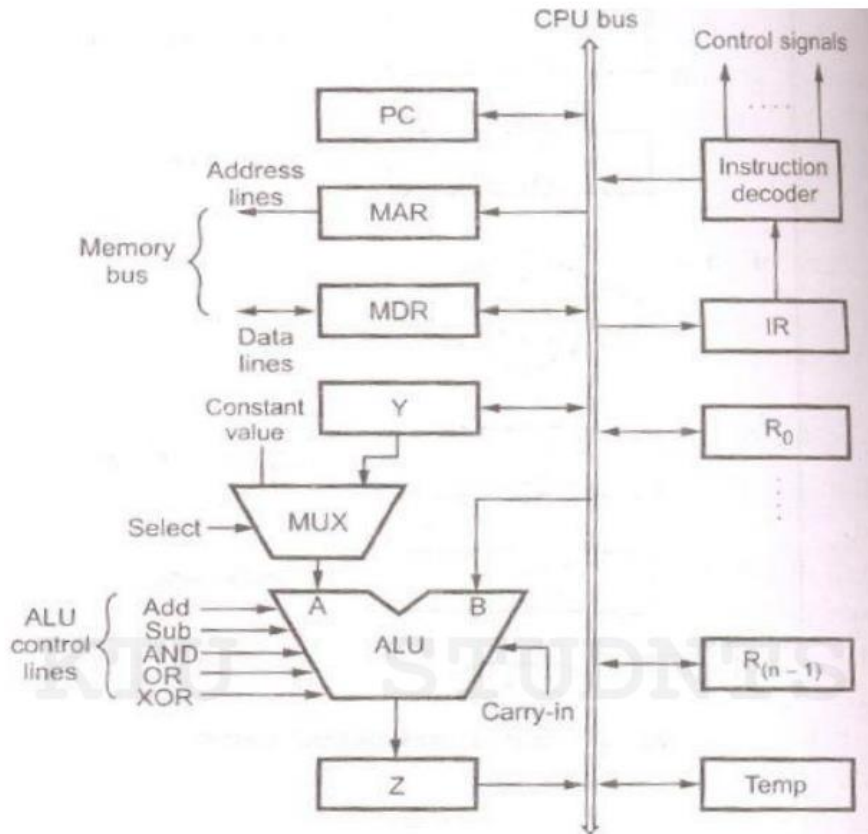


Fig. 3.2 Single bus organisation of processor

- The registers Y,Z and Temp are used only by the processor unit for temporary storage during the execution of some instructions.
- ☐ These registers are never used for storing data generated by one instruction for later use by another instruction.
- ☐ The programmer cannot access these registers.
- ☐ The IR and the instruction decoder are integral parts of the control circuitry in the processing unit.
- ☐ All other registers and the ALU are used for storing and manipulating data.
- ☐ The data registers, ALU and the interconnecting bus is referred to as data path.
- ☐ Register R₀ through R_(n-1) are the processor registers.

- □ The number and use of these register vary considerably from processor to processor.
-
- These registers include general purpose registers and special purpose registers such as stack pointer, index registers and pointers.
- ➤ These are 2 options provided for A input of the ALU.
- ➤ The multiplexer(MUX) is used to select one of the two inputs.
- ➤ It selects either output of Y register or a constant number as an A input for the ALU according to the status of the select input.
- ➤ It selects output of Y when select input is 1 (select Y) and it selects a constant number when select input is 0(select C) as an input A for the multiplier.
- ➤ The constant number is used to increment the contents of program counter.
- ➤ For the execution of various instructions processor has to perform one or more of the following basic operations:
- - a) Transfer a word of data from one processor register to the another or to the ALU.
 - b) perform the arithmetic or logic operations on the data from the processor registers and store the result in a processor register.
 - c) Fetch a word of data from specified memory location and load them into a processor register.
 - d) Store a word of data from a processor register into a specified memory location.

50.

b) Write the Sequence control signal required for the execution on single bus organization of data path, of the instruction **Add (R1),R2** with Appropriate explanations.

Ans :

1. PCout, MARin, Yin, select 4, Add, Zin
2. Zout, PCin, , WMFC
3. MDRout, IRin
4. R1out, MARin, Read
5. R2out, Yin, WMFC
6. MDRout, select Y, Add, Zin
7. Zout, R2 in

STEP 1

- The instruction fetch operation is initiated by loading the controls of the PC into the MAR.
- PC contents are also loaded into register Y and added constant number by activating select C input of multiplexer and add input of the ALU.
- By activating Zin signal result is stored in the register Z

STEP 2

- The contents of **register Z** are transferred to pc register by activating **Zout** and pcin signal.
- This completes the **PC** increment operation and **PC** will now point to next instruction,

STEP 3

- contents of **MDR** register are transferred to the instruction **register(IR)** of the processor
- The step 1 through 3 constitute the instruction fetch phase.
- At the beginning of step 4, the instruction decoder interprets the contents of the **IR**.

STEP 4

- Step 4, the contents of register **R1** are transferred to register MAR by activating **R1out** and MAR in signals.

STEP 5

- The contents of **register R2** are transferred to **register Y** by activating **R2out** and **Yin** signals

STEP 6

- **MDRout, select Y, Add and Zin** signals are activated to perform addition of contents of **register Y** and the contents of MDR.
- The result is stored in the **register Z**.

STEP 7

- The contents of register Z are transferred to **register R2** by activating **Zout** and **R2in** signals.....(5)

51.

Write the control sequence for the instruction DIV R1,[R2] in a three bus structure.

52. What is the significance of addressing modes in computer architecture.

53. Explain the concept of a single bus organization with help of a diagram. Write the control sequence for the instruction ADD [R1],[R2].